

PROJECT ETHER

Blueprint Técnico, Game Design, Economia e Implementação

Versão 2.0 — Especificação Base para Desenvolvimento

Status: Proposed / Implementation Baseline

Tipo: MMORPG 2D Mobile-First

Plataformas iniciais: Android / iOS

Backend candidato: C# / .NET

Cliente candidato: Godot

Persistência: PostgreSQL

Cache/coordenação: Redis

Infraestrutura: Linux / Docker

Arquitetura inicial: Modular Monolith + serviços auxiliares quando necessário

1. VISÃO DO PRODUTO

1.1 Conceito

PROJECT ETHER é um MMORPG 2D mobile-first baseado em:

- exploração;
- descoberta;
- progressão;
- combate em tempo real;
- economia dirigida pelos jogadores;
- interação social;
- profissões;
- equipamentos;

- consequências;
- mundo persistente.

A frase central do produto:

“Um mundo que continua existindo quando você fecha o jogo.”

O objetivo não é reproduzir Tibia.

O produto deve capturar algumas das propriedades que tornam MMORPGs clássicos interessantes — liberdade, descoberta, progressão, risco, economia e interação social — mas reconstruí-las para uma experiência moderna e adequada a celulares.

2. PRINCÍPIOS INVOLÁVEIS

Estas regras são consideradas princípios de produto.

P1 — Server authoritative

O cliente nunca possui autoridade sobre:

- posição definitiva;
- dano;
- XP;
- loot;
- inventário;
- dinheiro;
- cooldown;
- resultado de crafting;
- resultado de trade;
- conclusão de quest;
- estado de combate.

O cliente solicita.

O servidor valida e decide.

P2 — Sem autoplay de combate

O jogador não pode ativar um sistema que:

- procure criaturas;
- ataque automaticamente;
- utilize habilidades automaticamente;
- recolha loot automaticamente;
- percorra o mapa sozinho.

Automação de atividades administrativas/produtivas é permitida.

Automação de gameplay competitivo não é.

P3 — Sem pay-to-win

Não vender diretamente:

- dano;
- defesa;
- XP competitivo;
- equipamento superior;
- vantagens exclusivas de combate;
- acesso obrigatório a conteúdo competitivo.

Itens cosméticos e conveniência podem existir.

P4 — Mobile não é uma versão reduzida do PC

Toda mecânica deve ser projetada considerando:

- toque;
- tela pequena;
- sessões interrompidas;
- notificações;
- reconexão;
- bateria;
- temperatura;
- conectividade instável;

- uso com uma mão.

P5 — O mundo deve recompensar curiosidade

Não transformar todo conteúdo em:

“vá até o marcador X → mate Y → volte para Z”.

O jogador deve poder descobrir coisas por:

- observação;
- exploração;
- diálogo;
- mapas;
- pistas;
- comportamento dos NPCs;
- ambiente;
- rumores;
- experimentação.

P6 — Progressão não deve depender de grind artificial

Grind pode existir.

Mas não deve ser a única forma de progressão.

O jogador pode evoluir através de:

- combate;
- exploração;
- profissões;
- crafting;
- comércio;
- quests;
- social;
- descobertas;
- achievements;
- especialização.

3. PÚBLICO-ALVO

Primário

Jogadores que gostam de:

- MMORPGs clássicos;
- exploração;
- loot;
- progressão;
- economia;
- construção de personagem;
- socialização;
- descoberta;
- combate em tempo real.

Secundário

Jogadores mobile que:

- não gostam de autoplay;
- querem sessões curtas;
- mas ocasionalmente jogam por horas.

4. ESTRUTURA DE SESSÃO

O jogo deve funcionar em três escalas.

Micro

3–10 minutos

Exemplos:

- vender itens;
- comprar;
- coletar recursos;
- completar uma etapa;
- fazer crafting;

- conversar;
- explorar uma pequena área;
- participar de evento curto.

Normal

10–30 minutos

Exemplos:

- hunt;
- quest;
- dungeon;
- exploração;
- coleta;
- grupo.

Longa

30+ minutos

Exemplos:

- dungeon;
- boss;
- guild event;
- exploração profunda;
- expedição;
- cadeia de quests.

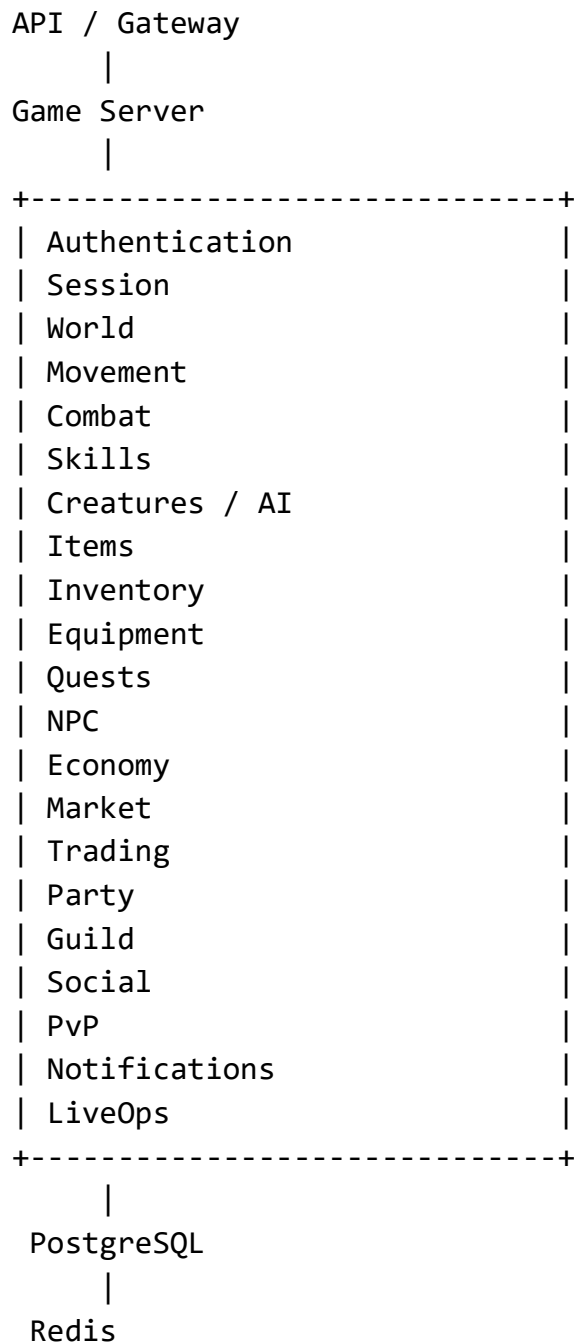
Nenhum sistema fundamental deve obrigar o jogador a entrar diariamente.

5. ARQUITETURA DO JOGO

5.1 Componentes

Mobile Client

|
| HTTPS / WebSocket
|



6. ESTRATÉGIA ARQUITETURAL

Inicialmente:

Modular Monolith.

Não começar com dezenas de microserviços.

O código deve possuir fronteiras modulares fortes.

Cada módulo deve possuir:

- domínio;
- aplicação;
- infraestrutura;
- contratos;
- eventos;
- testes.

A extração para serviços independentes poderá ocorrer posteriormente.

7. BACKEND

7.1 Stack

Runtime

.NET 10 LTS ou versão LTS vigente no início da implementação.

Linguagem

C#.

Banco

PostgreSQL.

Cache

Redis.

Comunicação

HTTPS para APIs.

WebSocket para gameplay em tempo real.

Containerização

Docker.

Observabilidade

OpenTelemetry.

Métricas:

- Prometheus;
- Grafana;

ou solução equivalente.

Logs estruturados.

Tracing distribuído preparado desde o início.

8. MÓDULOS

Core

- Identity
- Accounts
- Characters
- Sessions

Gameplay

- World
- Maps
- Movement
- Combat
- Skills
- Spells
- Creatures
- AI
- Items
- Inventory

- Equipment
- Loot

Content

- NPC
- Quests
- Dialogues
- Events
- Dungeons
- Achievements

Social

- Friends
- Messaging
- Party
- Guild
- Reputation
- Reports
- Blocking

Economy

- Currency
- Wallet
- Trading
- Marketplace
- Crafting
- Professions

Operations

- Notifications
- Analytics
- LiveOps
- Admin
- Audit
- Moderation

9. IDENTIDADE

Account

AccountId
Email / ExternalIdentity
CreatedAt
LastLoginAt
Status
Country
BirthDate / AgeVerificationState
TermsVersion
PrivacyVersion

Nunca armazenar senha em texto.

Preferir:

- Apple;
- Google;
- email/password;
- outros provedores posteriormente.

10. CHARACTER

CharacterId
AccountId
Name
Level
Experience
ClassId
SpecializationId
CurrentMapId
Position
Health
Mana

CreatedAt
LastActiveAt
Status

Estado adicional:

- atributos;
- skills;
- equipment;
- inventory;
- quests;
- reputation;
- achievements;
- professions;
- currencies;
- social relations.

11. ATRIBUTOS

Atributos base propostos:

Strength
Dexterity
Intelligence
Vitality
Willpower

Strength

Influência:

- dano físico;
- capacidade física;
- determinados equipamentos.

Dexterity

Influência:

- precisão;
- velocidade;
- evasão;
- ataques à distância.

Intelligence

Influência:

- poder mágico;
- mana;
- eficiência de habilidades mágicas.

Vitality

Influência:

- HP;
- resistência física.

Willpower

Influência:

- resistência mágica;
- regeneração;
- efeitos de controle.

Os efeitos exatos dos atributos devem ser centralizados em fórmulas versionadas.

12. LEVEL

Level é uma progressão vertical.

Cap inicial:

PRODUCT_DECISION_REQUIRED

Recomendação inicial para protótipo:

100 níveis.

Não implementar level 100 como obrigação de lançamento.

O sistema deve permitir expansão.

13. XP

Modelo:

`XPRequired(level)`

A curva não deve ser linear.

Modelo inicial:

$XP(level) = BaseXP * level^{1.65}$

Valores finais devem ser calibrados através de playtests.

Toda alteração da curva deve gerar:

`ProgressionFormulaVersion`

Não alterar fórmulas historicamente sem versionamento.

14. CLASSES

Versão inicial:

1. Warrior
2. Ranger
3. Arcanist

Classes representam identidade inicial.

Não devem bloquear completamente outras formas de jogar.

15. WARRIOR

Características:

- combate próximo;
- defesa;
- resistência;
- controle de espaço.

Recursos:

- HP elevado;
- defesa elevada;
- mobilidade moderada.

Armas:

- Sword;
- Axe;
- Mace;
- Shield.

16. RANGER

Características:

- distância;
- mobilidade;
- posicionamento;
- dano sustentado.

Armas:

- Bow;
- Crossbow;
- eventualmente armas especiais.

Dependência maior de:

- distância;
- terreno;

- posicionamento.

17. ARCANIST

Características:

- magia;
- controle;
- dano elemental;
- utilidade.

Recursos:

- Mana;
- efeitos;
- área.

18. SPECIALIZATION

Classe não determina todo o personagem.

Posteriormente o jogador escolhe especializações.

Exemplo:

Warrior:

- Guardian
- Berserker
- Duelist

Ranger:

- Hunter
- Scout
- Sharpshooter

Arcanist:

- Elementalist

- Occultist
- Spellbinder

As especializações devem modificar estilos de jogo, não simplesmente adicionar números maiores.

19. SKILLS

Skills independentes da classe.

Exemplos:

Sword

Axe

Mace

Bow

Shield

Magic

Mining

Woodcutting

Fishing

Cooking

Blacksmithing

Alchemy

Skill possui:

SkillId

CharacterId

Level

Experience

ProgressionVersion

20. EQUIPAMENTO

Slots:

Head
Chest
Legs
Feet
MainHand
OffHand
Ring1
Ring2
Amulet
Back

Possíveis expansões:

- Belt;
- Trinket;
- Cosmetic slots.

21. ITEM DEFINITION

Define o tipo do item.

ItemDefinitionId
Name
Category
SubCategory
Stackable
MaxStack
Weight
BaseValue
Rarity
RequiredLevel
RequiredSkill
BindPolicy
EquipmentSlot

22. ITEM INSTANCE

Representa o objeto individual.

ItemInstanceId
ItemDefinitionId
OwnerId
Quantity
Durability
Quality
Modifiers
CreatedAt
UpdatedAt
Bound
Origin

Itens que não precisam de individualização podem utilizar stacks agregados.

Itens especiais devem possuir identidade própria.

23. RARIDADE

Inicialmente:

- Common
- Uncommon
- Rare
- Epic
- Legendary

Raridade não significa automaticamente:

“mais forte”.

Pode representar:

- combinação de atributos;
- aparência;
- história;
- efeito;

- dificuldade de obtenção.

24. INVENTÁRIO

O inventário possui:

- slots;
- stacks;
- containers;
- equipamento;
- itens vinculados;
- itens negociáveis.

Operações devem ser transacionais.

Exemplo:

AddItem

RemoveItem

SplitStack

MergeStack

MoveItem

EquipItem

UnequipItem

DestroyItem

TransferItem

Nenhuma dessas operações pode permitir quantidade negativa.

25. PESO

Peso não será obrigatório no MVP.

PRODUCT_DECISION_REQUIRED

Recomendação:

Começar sem peso para reduzir fricção mobile.

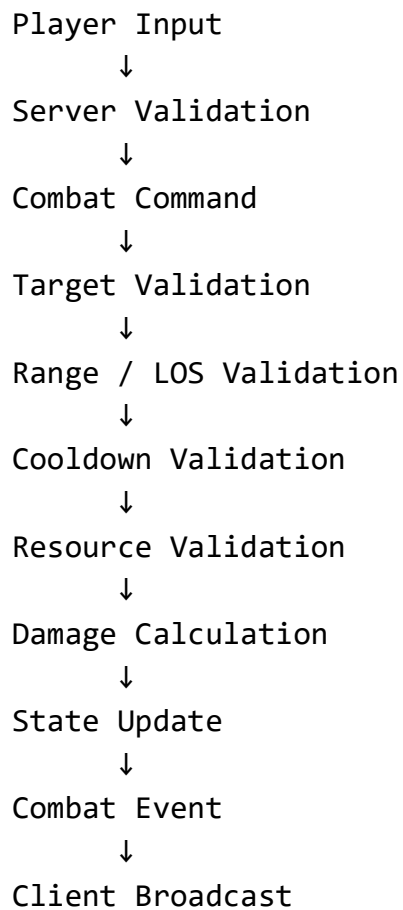
Adicionar posteriormente apenas se testes mostrarem que inventário perdeu profundidade.

26. COMBATE

Combate é:

- real-time;
- server-authoritative;
- baseado em posição;
- baseado em alvo;
- baseado em habilidades.

Fluxo:



27. DAMAGE

Modelo inicial:

```
RawDamage =  
BaseDamage  
+ WeaponPower  
+ AttributeContribution  
+ SkillContribution
```

Depois:

```
MitigatedDamage =  
RawDamage * DefenseMultiplier
```

Depois:

```
FinalDamage =  
MitigatedDamage * ResistanceMultiplier
```

Crítico:

```
CriticalDamage =  
FinalDamage * CriticalMultiplier
```

Valores exatos serão definidos no balanceamento.

28. DEFESA

Defesa não deve reduzir dano linearmente indefinidamente.

Modelo inicial:

```
DamageMultiplier =  
100 / (100 + EffectiveDefense)
```

Exemplo:

100 de defesa:

$$100 / 200 = 50\%$$

200:

$$100 / 300 = 33.3\%$$

Isso reduz o risco de defesa tornar dano irrelevante em níveis altos.

29. RESISTÊNCIAS

Tipos iniciais:

Physical
Fire
Ice
Lightning
Poison
Arcane

Cada habilidade pode possuir:

DamageType
BasePower
Scaling
Penetration
Area
Cooldown
ResourceCost
Duration
StatusEffects

30. STATUS EFFECTS

Sistema genérico:

Burn
Poison
Bleed
Freeze
Slow
Root
Silence
Stun
Shield
Regeneration
Vulnerability

Todo efeito possui:

EffectId
Source
Target
Duration
Stacks
MaxStacks
Strength
StartedAt
ExpiresAt

31. COOLDOWNS

Cooldown pertence ao servidor.

O cliente recebe:

AbilityId
StartedAt
Duration

Remaining

O cliente apenas apresenta.

32. POSICIONAMENTO

O servidor valida:

- velocidade máxima;
- aceleração;
- teleporte;
- colisão;
- obstáculos;
- zonas proibidas;
- alcance;
- distância;
- line of sight.

Tentativas impossíveis geram:

`MovementValidationFailure`

Repetição excessiva pode gerar:

`SuspiciousMovementEvent`

33. MOVIMENTO

Comandos:

`MoveRequest`

`StopRequest`

`InteractRequest`

`AttackRequest`

`AbilityRequest`

O cliente envia intenção.

O servidor determina estado válido.

34. MAPA

O mundo é dividido em mapas.

Cada mapa possui:

MapId

Name

Width

Height

ZoneType

RecommendedLevel

WeatherProfile

MusicProfile

Tiles:

Walkable

BlocksMovement

BlocksSight

GroundType

Height

SpeedModifier

DamageModifier

InteractionType

35. WORLD

O lançamento não deve possuir um continente gigantesco vazio.

Modelo:

Capital

- |— Farmlands
- |— Forest
- |— Mines
- |— Ruins
- |— Swamp
- |— Wilderness

Poucas regiões.

Alta densidade de conteúdo.

36. EXPLORAÇÃO

O mapa deve conter:

- atalhos;
- áreas secretas;
- NPCs escondidos;
- eventos;
- recursos;
- criaturas raras;
- puzzles;
- ruínas;
- cavernas;
- caminhos alternativos;
- histórias ambientais.

Nem todo segredo deve aparecer no mapa.

37. CREATURE DEFINITION

CreatureDefinitionId

Name

Level

Health

Mana

Armor
Resistances
ExperienceReward
LootTableId
AiProfileId
MovementSpeed
AggroRange
AttackRange
RespawnProfile

38. AI

Estados:

Idle
Patrol
Investigate
Chase
Attack
Flee
Assist
Return
Dead

Bosses podem utilizar:

Phase1
Phase2
Phase3
Enrage

39. AGGRO

Critérios possíveis:

- distância;

- dano;
- cura;
- ameaça;
- proximidade;
- comportamento específico.

O sistema de ameaça deve ser configurável por criatura.

40. LOOT

Loot é decidido pelo servidor.

LootTable

- └ Item
- └ Chance
- └ MinQuantity
- └ MaxQuantity
- └ Conditions
- └ Weight

Não utilizar aleatoriedade no cliente.

Random deve ser executado server-side.

41. QUESTS

Tipos:

Exploration

Combat

Collection

Crafting

Dialogue

Investigation

Escort

Puzzle

Discovery

WorldEvent

Quest possui estado:

Available

Active

ObjectiveProgress

Completed

Failed

Expired

42. QUEST ENGINE

Objetivos devem ser data-driven.

Exemplo:

```
{  
  "type": "KillCreature",  
  "target": "forest_wolf",  
  "required": 5  
}
```

Outros:

CollectItem

ReachLocation

TalkToNPC

CraftItem

DiscoverLocation

UseAbility

CompleteQuest

43. QUEST BRANCHING

Algumas quests podem gerar escolhas.

Exemplo:

Ajudar aldeia

```
|
+-- Defender aldeia
|
+-- Investigar invasores
```

As escolhas podem alterar:

- reputação;
- NPCs;
- recompensas;
- acesso;
- diálogos;
- eventos.

Não transformar toda quest em árvore complexa.

44. NPC

NPC pode possuir:

Identity

Routine

Dialogue

Shop

QuestProvider

QuestTurnIn

Faction

ReputationRules

Memory

Rotinas:

Sleep
Work
Travel
Eat
Socialize
Guard
Trade

Isso ajuda o mundo a parecer vivo.

45. MORTE

Morte deve possuir consequência.

Porém:

não copiar a severidade de sistemas hardcore.

Modelo inicial:

- respawn em local seguro;
- perda parcial de recursos;
- possível perda de durabilidade;
- risco de perder parte de loot não protegido;
- cooldown/recovery curto.

Penalidade final:

PRODUCT_DECISION_REQUIRED

46. DISCONNECT

Disconnect não equivale automaticamente a morte.

O servidor mantém:

CharacterState
Position

CombatState

HP

Mana

Inventory

Threat

Se o jogador desconectar:

- personagem permanece por curto período em situação controlada;
- proteção pode existir em cidades;
- combate deve seguir regras específicas;
- reconexão recupera estado server-side.

47. RECONNECTION

Cliente deve suportar:

ConnectionLost

Reconnecting

Reconnected

StateSynchronizing

Ready

Após reconexão:

1. autenticar;
2. validar sessão;
3. recuperar personagem;
4. sincronizar posição;
5. sincronizar entidades próximas;
6. sincronizar inventário;
7. sincronizar combate;
8. sincronizar quests;
9. voltar ao gameplay.

48. ECONOMIA

Moeda principal:

Gold

Moeda premium:

Ether ou nome final ainda `PRODUCT_DECISION_REQUIRED`.

A economia deve possuir:

Sources

- monstros;
- quests;
- venda;
- eventos;
- atividades.

Sinks

- marketplace tax;
- NPC services;
- crafting;
- repair;
- travel;
- guild;
- cosmetics;
- outros serviços.

49. ECONOMIA ANTIFLAÇÃO

Não criar uma economia em que:

Gold generation >> Gold sinks

Monitorar:

GoldCreated
GoldDestroyed
GoldCirculation
AverageWallet
MedianWallet
Top1%
Top10%
InflationRate

Alertas automáticos quando os indicadores ultrapassarem thresholds.

50. MARKETPLACE

Marketplace funciona offline.

Jogador pode:

CreateListing
CancelListing
BuyListing
SearchListings
FilterListings

Listing:

ListingId
SellerId
Item
Quantity
UnitPrice
CreatedAt
ExpiresAt
Status

Estados:

Active
Sold
Cancelled

Expired

51. MARKETPLACE TRANSACTION

Compra:

BEGIN TRANSACTION

Validate listing
Lock listing
Validate buyer balance
Debit buyer
Transfer item
Credit seller
Apply tax
Mark listing sold
Write audit
Commit

Se qualquer etapa falhar:

ROLLBACK

Nenhum item ou moeda pode ser duplicado.

52. PLAYER TRADE

Trade direto:

TradeRequested
TradeAccepted
ItemsLocked
CurrenciesLocked
TradeConfirmed
TradeCompleted

O trade só executa depois que ambos confirmarem.

Alterar qualquer item depois da primeira confirmação deve invalidar a confirmação.

53. CRAFTING

Crafting não é apenas:

material + botão = item.

Pode envolver:

Recipe

Materials

Profession

Station

Time

Quality

Skill

OptionalModifiers

54. PROFISSÕES

Inicialmente:

Blacksmith

Alchemy

Cooking

Fishing

Mining

Woodcutting

Nem todas precisam estar no MVP.

Profissões são progressão horizontal.

55. QUALITY

Crafting pode gerar:

Poor

Normal

Fine

Superior

Masterwork

Qualidade pode afetar:

- atributos;
- durabilidade;
- aparência;
- valor.

Não necessariamente todos.

56. OFFLINE ACTIVITIES

Permitidas:

- crafting;
- produção;
- market;
- expedições;
- pesquisa;
- tarefas administrativas.

Não permitidas:

- combate autônomo;
- farm automático;
- PvP automático;
- exploração automática.

57. PARTY

Party:

Create

Invite

Accept

Leave

Kick

Disband

Estado:

Leader

Member

Distribuição:

- XP compartilhada;
- loot configurável;
- regras anti-abuso.

Não obrigar composição:

Tank + Healer + DPS

58. GUILD

Guild possui:

GuildId

Name

Leader

Members

Level

Experience

Storage

Projects

Reputation
Permissions

Ranks:

Leader
Officer
Member
Recruit

Permissões configuráveis.

59. GUILD FEATURES

Inicialmente:

- guild chat;
- membros;
- storage;
- announcements;
- projects;
- progression.

Futuro:

- guild expeditions;
- territory;
- guild bosses;
- wars;
- guild events.

60. PVP

PvE deve estar completo antes de depender de PvP para retenção.

Inicialmente:

- duels;
- arenas;
- PvP zones;
- events.

Open-world PvP:

PRODUCT_DECISION_REQUIRED

Recomendação:

não implementar no primeiro lançamento.

61. REPUTATION

Reputação pode existir com:

- cidades;
- facções;
- guilds;
- NPCs.

Exemplo:

Hostile

Unfriendly

Neutral

Friendly

Honored

Revered

Reputação pode liberar:

- quests;
- lojas;
- cosméticos;
- acesso;
- diálogos.

62. SOCIAL

Obrigatório:

- friends;
- private messages;
- party;
- guild;
- trade;
- block;
- report.

Opcional:

- voice;
- emojis avançados;
- social profiles.

63. MODERAÇÃO

Report categories:

Harassment

Spam

Scam

Cheating

Botting

InappropriateName

Other

Admin actions:

Warning

Mute

TemporaryBan

PermanentBan

TradeRestriction

ChatRestriction

Toda ação administrativa gera:

`AdminAuditEvent`

64. MOBILE UX

Controles

Suportar:

Tap-to-move

Toque no destino.

Virtual joystick

Movimentação contínua.

Contextual interaction

O botão muda conforme contexto:

Talk

Loot

Open

Attack

Gather

Enter

65. COMBATE MOBILE

HUD:

Target

HP / Status

Movement

Abilities

Action

Priorizar:

- poucos botões;
- grandes áreas de toque;
- feedback visual;
- feedback sonoro;
- haptics quando apropriado.

66. UMA MÃO / DUAS MÃOS

Uma mão:

- movimento;
- ataque básico;
- ações simples.

Duas mãos:

- habilidades;
- movimentação;
- target switching;
- microgerenciamento.

O jogo não deve exigir precisão extrema de touchscreen.

67. INVENTÁRIO MOBILE

Não criar uma interface semelhante a um desktop reduzido.

Priorizar:

- categorias;
- busca;
- filtros;

- stack;
- equipamento;
- comparação;
- ações contextuais.

Exemplo:

[Equipment]
[Consumables]
[Materials]
[Quest]
[Misc]

68. NOTIFICAÇÕES

Notificações devem ter valor.

Exemplos:

- venda concluída;
- crafting concluído;
- expedição concluída;
- guild event;
- mensagem importante;
- evento começando.

Não:

“Você não entrou hoje!”

69. RETENÇÃO

Não usar:

- energia obrigatória;
- daily obrigatório;
- punição por ausência;
- FOMO excessivo;

- autoplay.

Retenção deve vir de:

Curiosity

Progress

Goals

Relationships

Economy

Mastery

Collection

Discovery

70. MONETIZAÇÃO

Modelo:

Free-to-play.

Produtos possíveis:

Cosméticos

- outfits;
- mounts;
- pets cosméticos;
- efeitos;
- emotes;
- títulos;
- decoração.

Convenience

Somente quando não causar vantagem competitiva significativa.

Battle/Season Pass

Pode fornecer:

- cosméticos;

- recursos visuais;
- recompensas de progressão não competitivas.

Subscription

Possível posteriormente.

71. MONETIZAÇÃO PROIBIDA

Não vender diretamente:

+20% damage
+30% defense
exclusive best weapon
exclusive best armor
guaranteed competitive advantage

Evitar loot boxes inicialmente.

72. PREMIUM CURRENCY

Se existir:

Real Money
↓
Premium Currency
↓
Cosmetics / Services

Nunca:

Real Money
↓
Competitive Power

73. ANALYTICS

Eventos obrigatórios:

AppInstalled
AccountCreated
CharacterCreated
TutorialStarted
TutorialCompleted
FirstMovement
FirstCombat
FirstKill
FirstLoot
FirstLevelUp
FirstQuest
FirstDeath
FirstTrade
FirstMarketOrder
FirstGuild
FirstPurchase
SessionStarted
SessionEnded
Disconnect
Reconnect

74. FUNIL

Medir:

Install
↓
Account
↓
Character
↓
First movement
↓
First combat
↓

First kill
↓
First loot
↓
First quest
↓
First level
↓
First social interaction
↓
Return

Qualquer queda significativa deve gerar investigação.

75. RETENÇÃO

Monitorar:

D1
D3
D7
D14
D30
D60
D90

Separar por:

- plataforma;
- versão;
- região;
- device;
- origem;
- cohort;
- classe;
- comportamento.

76. PERFORMANCE

Targets iniciais:

Startup

Ideal:

< 5 segundos

em dispositivo de referência.

Reconnect

Ideal:

< 5 segundos

quando possível.

Gameplay

Objetivo:

60 FPS

em dispositivos de referência.

Fallback:

30 FPS

sem comprometer jogabilidade.

77. DISPOSITIVOS

Definir tiers:

Low

Medium

High

Cada tier possui:

- resolução;
- efeitos;
- quantidade de entidades;
- partículas;
- sombras;
- distância de renderização.

78. SERVER TICK

O servidor não precisa atualizar todo o mundo na mesma frequência.

Exemplo inicial:

Movement: 10-20 Hz

Combat: 10-20 Hz

AI: 5-10 Hz

Non-critical simulation: lower frequency

Valores finais serão determinados por profiling.

79. INTEREST MANAGEMENT

Cliente não recebe o mundo inteiro.

Servidor envia:

- jogadores próximos;
- criaturas próximas;
- NPCs relevantes;
- eventos relevantes;
- objetos relevantes.

Conceito:

InterestRadius

Pode ser dividido em:

Immediate

Nearby

Peripheral

80. WORLD INSTANCE

O mundo pode utilizar instâncias para:

- dungeons;
- eventos;
- conteúdo privado;
- bosses.

Overworld continua compartilhado.

81. DATABASE PRINCIPAL

Entidades principais:

Account

Character

CharacterAttribute

CharacterSkill

CharacterEquipment

Inventory

ItemDefinition

ItemInstance

Map

MapTile

CreatureDefinition

CreatureInstance

NpcDefinition

QuestDefinition

QuestState

Guild

GuildMember
Party
Trade
MarketListing
Wallet
CurrencyTransaction
CraftingRecipe
CraftingJob
Profession
Achievement
Notification
AuditEvent
AdminAction

82. TRANSAÇÕES FINANCEIRAS

Toda alteração monetária gera:

CurrencyTransaction

Nunca depender apenas do saldo atual.

Exemplo:

TransactionId
CharacterId
Currency
Amount
Type
ReferenceId
CreatedAt
BalanceAfter

Isso permite auditoria.

83. INVENTORY LEDGER

Itens de alto valor devem possuir histórico.

ItemCreated
ItemTransferred
ItemSplit
ItemMerged
ItemDestroyed
ItemEquipped
ItemUnequipped
ItemSold

Isso permite investigação de duplicações.

84. IDEMPOTÊNCIA

Comandos sensíveis possuem:

CommandId

O servidor deve impedir execução duplicada.

Especialmente:

- compra;
- venda;
- trade;
- reward;
- purchase;
- crafting completion;
- quest reward.

85. EVENT BUS

Eventos internos:

CharacterCreated
CharacterMoved
CombatStarted
DamageApplied
CreatureKilled
LootGenerated
ItemTransferred
ExperienceGranted
LevelUp
QuestCompleted
TradeCompleted
MarketOrderCreated
MarketOrderFilled
CharacterDied
GuildCreated
GuildMemberJoined
PurchaseCompleted

Eventos não devem ser utilizados como substituto indiscriminado de transações.

86. COMANDO VS EVENTO

Comando:

“Faça X.”

Evento:

“X aconteceu.”

Exemplo:

AttackCreatureCommand

gera:

AttackExecuted
DamageApplied

Não permitir que o cliente envie:

DamageApplied

87. API

Exemplos:

POST /auth/login

POST /characters

GET /characters

POST /sessions

POST /market/listings

POST /market/purchase

POST /trades

POST /guilds

GET /quests

Gameplay:

WebSocket

88. GAMEPLAY PROTOCOL

Mensagens:

MoveRequest

MoveAccepted

MoveRejected

AttackRequest

AttackAccepted

AttackRejected

AbilityRequest

AbilityAccepted

AbilityRejected

EntitySpawn

EntityDespawn

EntityUpdate

CombatEvent

DamageEvent

DeathEvent

LootEvent

O protocolo deve ser versionado.

89. VERSIONAMENTO

Versões:

ProtocolVersion

GameBalanceVersion

ContentVersion

DatabaseSchemaVersion

Um personagem criado sob uma fórmula antiga deve continuar consistente quando necessário.

90. SAVE MODEL

Não depender de “save game” tradicional.

O estado persistente é continuamente salvo.

Eventos importantes:

CharacterCreated

LevelUp

ItemAcquired

ItemTransferred

QuestCompleted
TradeCompleted
Death
Purchase

91. CONCORRÊNCIA

Operações críticas devem utilizar:

- transaction;
- row locking;
- optimistic concurrency;
- unique constraints;
- idempotency keys.

Especialmente:

- marketplace;
- trade;
- wallet;
- inventory.

92. ANTI-DUPLICATION

Camadas:

1. server authority;
2. transaction;
3. unique IDs;
4. item ownership;
5. ledger;
6. idempotency;
7. audit;
8. anomaly detection.

93. BOT DETECTION

Não banir apenas baseado em um indicador.

Signals:

- repetição perfeita;
- padrões temporais;
- movimentação;
- frequência;
- sessões;
- input timing;
- economia;
- comportamento impossível.

Sistema deve gerar:

`BotSuspicionScore`

para análise.

Não deve banir automaticamente apenas por score baixo/médio.

94. SECURITY

Obrigatório:

- TLS;
- token rotation;
- session expiration;
- rate limiting;
- request validation;
- server-side validation;
- anti-replay;
- secure storage;
- audit logs.

95. ADMIN PANEL

Funcionalidades:

Players

- procurar;
- visualizar;
- bloquear;
- banir;
- silenciar;
- investigar.

Items

- consultar;
- criar;
- remover;
- transferir com auditoria.

Economy

- consultar;
- transações;
- inflation;
- suspicious activity.

Content

- quests;
- creatures;
- NPCs;
- items;
- events.

World

- maps;
- spawns;
- events.

96. ADMIN SECURITY

Admin actions precisam de:

AdminId
Action
Target
Before
After
Timestamp
Reason

Ações sensíveis podem exigir:

- segundo fator;
- confirmação;
- reason obrigatório.

97. LIVE OPS

Eventos podem ser configurados por dados.

EventDefinition
StartAt
EndAt
Zones
Rewards
Modifiers
Requirements

Exemplos:

- invasão;
- festival;
- boss;
- resource surge;
- seasonal event.

98. CONTENT DATA-DRIVEN

Evitar hardcode de:

- criaturas;
- itens;
- quests;
- loot;
- NPCs;
- receitas;
- eventos.

O máximo possível deve ser configurável através de dados.

Isso reduz custo de produção.

99. CONTENT PIPELINE

Fluxo:

Design

↓

Data Definition

↓

Validation

↓

Automated Tests

↓

Staging

↓

Playtest

↓

Balance

↓

Production

100. TESTES

Unit

- fórmulas;
- regras;
- domínio.

Integration

- PostgreSQL;
- Redis;
- transactions.

Contract

- API;
- WebSocket.

Gameplay

- combate;
- loot;
- quests;
- movement.

Load

- concurrent players;
- combat;
- marketplace.

Security

- tampering;
- replay;
- duplication;
- invalid commands.

101. DETERMINISMO

Fórmulas importantes devem ser determinísticas.

Exemplo:

DamageCalculator
ExperienceCalculator
LootCalculator
CraftingCalculator

recebem inputs explícitos.

Isso permite testes como:

Given X
When Y
Then Z

102. BALANCE ENGINE

Não espalhar números mágicos pelo código.

Errado:

```
damage *= 1.37;
```

Preferir:

```
BalanceConfiguration
```

com versão.

Exemplo:

CombatFormulaVersion = 1

103. CONFIGURAÇÃO

Separar:

Code

Regras estruturais.

Data

Conteúdo.

Configuration

Valores ajustáveis.

LiveOps

Eventos temporários.

104. LOGGING

Logs estruturados.

Exemplo:

```
{  
  "event": "MarketPurchase",  
  "characterId": "...",  
  "listingId": "...",  
  "amount": 1500,  
  "currency": "Gold"  
}
```

Nunca registrar:

- senha;
- token;
- informações sensíveis desnecessárias.

105. OBSERVABILIDADE

Métricas:

ConnectedPlayers
ActivePlayers
RequestsPerSecond
CombatEventsPerSecond
TickDuration
DatabaseLatency
RedisLatency
WebSocketConnections
ReconnectRate
CrashRate

Alertas:

- tick degradation;
- database latency;
- abnormal disconnect;
- economy anomaly;
- memory;
- CPU.

106. SHARDING

Não implementar inicialmente.

Arquitetura deve permitir futuramente:

World
├─ Shard 1
└─ Shard 2

└─ Shard 3

Mas não pagar a complexidade antes de existir necessidade.

107. WORLD SCALING

Inicialmente:

1 logical world

Posteriormente:

Region
└─ Zone
 └─ Instance

108. MOBILE ACCOUNT RECOVERY

Suportar:

- Google;
- Apple;
- email.

A conta não deve depender exclusivamente do dispositivo.

109. TUTORIAL

Tutorial deve ser curto.

Objetivos:

Move

↓

Interact

↓

Combat

↓

Loot

↓

Equipment

↓

Quest

↓

Explore

O jogador deve entrar no mundo rapidamente.

Não criar 30 minutos de tutorial.

110. PRIMEIROS 30 MINUTOS

Devem apresentar:

- mundo;
- combate;
- progressão;
- loot;
- equipamento;
- NPC;
- quest;
- descoberta;
- social;
- economia básica.

Não necessariamente todos de forma explícita.

111. FIRST EXPERIENCE

O jogador deve sentir:

“Tem alguma coisa ali. Quero descobrir.”

Mais importante que:

“Preciso completar minha daily.”

112. MVP

O MVP precisa conter somente o necessário para provar o core loop.

Obrigatório

- login;
- character;
- world;
- movement;
- 1 cidade;
- 1 região externa;
- 3–5 criaturas;
- combate;
- loot;
- inventory;
- equipment;
- XP;
- levels;
- 1 classe inicialmente;
- quests;
- NPC;
- death;
- reconnect;
- persistence;
- 2 players simultâneos;
- server authority.

113. MVP NÃO CONTÉM

- housing;

- territory;
- guild wars;
- open PvP;
- mounts;
- pets complexos;
- dezenas de profissões;
- dezenas de classes;
- marketplace avançado;
- centenas de criaturas;
- grande continente;
- sistema de transmog complexo;
- battle pass;
- monetização definitiva.

114. VERTICAL SLICE

O primeiro marco técnico deve permitir:

Login



Create Character



Enter World



Move



Find Creature



Attack



Kill



Loot



Equip



Gain XP



Level Up



Quest
↓
Die
↓
Respawn
↓
Logout
↓
Login
↓
State Restored

Com dois jogadores simultaneamente:

Player A
↕
Game Server
↕
Player B

115. VERTICAL SLICE ACCEPTANCE CRITERIA

Passa somente se:

- dois jogadores conseguem entrar;
- ambos conseguem se movimentar;
- ambos enxergam entidades;
- combate funciona;
- dano é validado pelo servidor;
- criatura morre;
- loot aparece;
- inventory persiste;
- XP persiste;
- level persiste;
- quest persiste;
- morte funciona;

- reconnect funciona;
- logout/login restaura estado;
- cliente não consegue fabricar dano;
- cliente não consegue fabricar item;
- cliente não consegue fabricar dinheiro;
- nenhuma operação crítica duplica recursos.

116. ROADMAP

Phase 0 — Foundation

- repository;
- CI/CD;
- architecture;
- database;
- authentication;
- logging;
- telemetry;
- Docker;
- test framework.

Phase 1 — Vertical Slice

- world;
- movement;
- combat;
- creature;
- loot;
- inventory;
- persistence.

Phase 2 — Core MMORPG

- classes;
- skills;
- abilities;
- quests;
- NPC;

- progression;
- more creatures.

Phase 3 — World

- additional maps;
- exploration;
- secrets;
- dungeons;
- events.

Phase 4 — Economy

- crafting;
- professions;
- marketplace;
- trading;
- economy telemetry.

Phase 5 — Social

- party;
- guild;
- friends;
- messaging;
- reputation.

Phase 6 — PvP

- duels;
- arenas;
- zones;
- events.

Phase 7 — LiveOps

- events;
- seasonal content;
- analytics;
- moderation;

- admin.

Phase 8 — Monetization

- cosmetics;
- premium;
- pass;
- subscription if justified.

Phase 9 — Production

- performance;
- security;
- load;
- store compliance;
- crash monitoring;
- release pipeline.

117. DEFINITION OF DONE

Uma feature não é “pronta” somente porque compila.

Ela precisa possuir:

Domain rule

Application flow

Persistence

Validation

Authorization

Tests

Telemetry

Logging where appropriate

Error handling

Concurrency handling

Documentation

Quando aplicável:

Migration
API contract
Client implementation
Admin support
Analytics

118. SPEC GAP PROTOCOL

Quando uma regra estiver ausente:

SPEC_GAP

Quando houver duas regras conflitantes:

SPEC_CONFLICT

Quando for necessária uma decisão de produto:

PRODUCT_DECISION_REQUIRED

O agente de desenvolvimento **não deve simplesmente inventar** uma regra com impacto estrutural.

119. EXEMPLO

Se o agente encontrar:

“O jogador perde XP ao morrer?”

Ele não deve escolher silenciosamente.

Registrar:

PRODUCT_DECISION_REQUIRED:
Death XP penalty undefined.
Impact:

- progression
- economy
- retention
- PvP
- risk/reward

120. PRINCÍPIO DE IMPLEMENTAÇÃO

A ordem correta é:

Função
↓
Regra
↓
Modelo
↓
Contrato
↓
Teste
↓
Implementação
↓
Telemetry

Não:

Código
↓
Descobrir regra depois

121. PRINCÍPIO DE GAME DESIGN

Não adicionar sistemas apenas porque MMORPGs tradicionais possuem esses sistemas.

Cada sistema precisa responder:

1. Qual comportamento cria?
2. Qual diversão proporciona?
3. Qual problema resolve?
4. Como interage com os outros sistemas?
5. Qual custo de manutenção possui?
6. Qual risco econômico cria?
7. Como funciona no mobile?

Se não houver resposta convincente:

não entra.

122. PRINCÍPIO DE ECONOMIA

Todo recurso infinito precisa possuir um sink.

Todo sink relevante precisa possuir justificativa.

Toda moeda deve possuir:

Source

Sink

Velocity

Distribution

Inflation Risk

123. PRINCÍPIO DE CONTEÚDO

É preferível:

20 criaturas interessantes

a:

200 criaturas genéricas.

Preferível:

10 quests memoráveis

a:

200 quests “mate 10 lobos”.

124. PRINCÍPIO DE MUNDO

O mundo não deve existir somente para transportar o jogador entre quests.

Cada região deve possuir:

- identidade;
- recursos;
- perigos;
- criaturas;
- histórias;
- economia;
- descobertas;
- motivo para retornar.

125. PRINCÍPIO DE SOCIAL

Social não deve ser apenas:

chat global.

Interação deve gerar dependência saudável:

Crafting

Trading

Party

Guild

Exploration

Events

Economy

O jogador deve encontrar razões naturais para conhecer outros jogadores.

126. PRINCÍPIO DE MONETIZAÇÃO

A pergunta não é:

“Como extrair o máximo de dinheiro?”

A pergunta é:

“Como monetizar uma experiência que o jogador considera valiosa sem destruir os sistemas que tornam o jogo interessante?”

127. PRIMEIRAS DECISÕES QUE DEVEM SER TESTADAS

Ainda não congelar:

Combat

- velocidade;
- TTK;
- quantidade de habilidades;
- alcance;
- cooldown.

Progression

- XP curve;
- level cap;
- skill progression.

Death

- penalidade;
- recovery.

Economy

- gold generation;
- sinks;
- item value.

Mobile

- joystick vs tap;
- câmera;
- HUD;
- tamanho de botões.

Content

- tamanho das regiões;
- densidade;
- frequência de descoberta.

Esses elementos devem ser calibrados através de protótipos.

128. PROTÓTIPO 0

Antes de construir o MMORPG completo:

Criar uma pequena arena.

Contendo:

- 1 player
- 1 creature
- 1 weapon
- 1 ability
- 1 map
- 1 loot
- 1 inventory
- 1 level

Objetivo:

validar se o combate e movimento são divertidos.

Se isso não funcionar:

não adicionar guild, marketplace, crafting ou monetização.

129. PROTÓTIPO 1

Adicionar:

2 players
3 creatures
1 NPC
1 quest
equipment
XP
death
reconnect

Objetivo:

validar o core loop.

130. PROTÓTIPO 2

Adicionar:

economy
crafting
market
social

Objetivo:

validar a segunda camada de retenção.

131. PROTÓTIPO 3

Adicionar:

guild
events
dungeons
advanced content

Objetivo:

validar retenção de médio/longo prazo.

132. PROTÓTIPO 4

Somente depois:

monetization
live ops
seasonal systems

133. PRIMEIRA ARQUITETURA DE REPOSITÓRIO

/project-ether

/src

/Ether.Domain
/Ether.Application
/Ether.Infrastructure
/Ether.Api
/Ether.GameServer
/Ether.Worker
/Ether.Contracts

```
/tests
  /Ether.Domain.Tests
  /Ether.Application.Tests
  /Ether.Infrastructure.Tests
  /Ether.Api.Tests
  /Ether.GameServer.Tests
  /Ether.Architecture.Tests
```

```
/client
  /Ether.Game
```

```
/tools
  /ContentTools
  /AdminTools
```

```
/docs
  /architecture
  /game-design
  /protocol
  /economy
  /balance
```

134. DOMAIN BOUNDARIES

O domínio deve evitar dependências de:

- HTTP;
- PostgreSQL;
- Redis;
- Godot;
- framework de UI.

Domain deve conhecer somente regras.

135. APPLICATION

Application coordena:

Commands
Queries
Handlers
Transactions
Authorization
Domain services

136. INFRASTRUCTURE

Responsável por:

- PostgreSQL;
- Redis;
- messaging;
- external providers;
- storage;
- telemetry;
- notifications.

137. GAME SERVER

Responsável por:

- sessions;
- world state;
- real-time commands;
- entity visibility;
- movement;
- combat tick;
- synchronization.

Não deve conter regra de negócio duplicada.

138. CLIENT

Cliente deve ser tratado como:

apresentação + input + prediction visual quando apropriado.

Nunca como autoridade.

139. CLIENT PREDICTION

Pode existir futuramente para:

- movimento;
- animação;
- efeitos.

Mas o servidor sempre corrige estado inválido.

140. DATA OWNERSHIP

Cada módulo deve possuir claramente seus dados.

Exemplo:

Inventory owns item ownership.

Wallet owns currency balance.

Quest owns quest state.

Guild owns membership.

Não criar tabelas “Deus”.

141. CROSS-MODULE OPERATIONS

Operações complexas utilizam:

- application orchestration;
- domain events;
- transactions quando necessário.

Exemplo:

KillCreature

↓

Combat

↓

Experience

↓

Loot

↓

Inventory

↓

Analytics

142. GAME STATE

Estados críticos devem possuir state machines explícitas.

Exemplo de criatura:

Spawned

↓

Idle

↓

Alerted

↓

Combat

↓

Dead

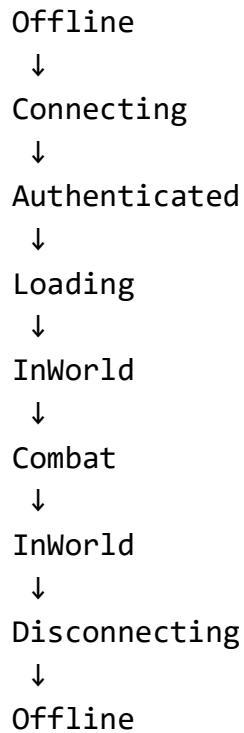
↓

Respawning

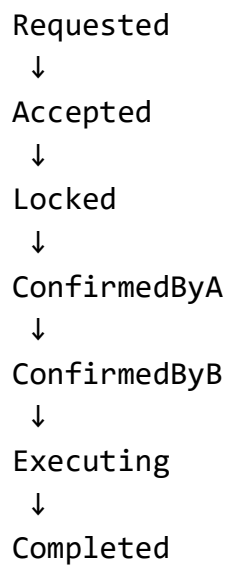
↓

Spawned

143. PLAYER STATE



144. TRADE STATE



Qualquer alteração antes de `Executing` pode invalidar confirmações.

145. MARKET STATE

Draft

↓

Active

↓

Sold

Alternativamente:

Active → Cancelled

Active → Expired

146. QUEST STATE

Available

↓

Active

↓

Progressing

↓

Completed

Com exceções:

Active → Failed

Active → Expired

147. CRAFTING STATE

Queued

↓

InProgress

↓

Completed

ou:

Queued → Cancelled

InProgress → Failed

148. ECONOMIC AUDIT

Toda movimentação importante de:

- gold;
- premium currency;
- item raro;
- equipamento raro;

deve ser rastreável.

Consulta:

Who

What

When

From

To

Why

Reference

149. GAME MASTER TOOLS

Posteriormente:

- teleport;
- inspect;

- spawn;
- grant;
- revoke;
- mute;
- ban;
- event activation.

Tudo auditado.

150. COMPLIANCE

Antes do lançamento:

- Apple App Store;
- Google Play;
- privacy;
- age rating;
- purchase policies;
- virtual currency;
- refund;
- regional requirements.

Para o Brasil:

- adequar classificação e requisitos aplicáveis;
- revisar regras específicas para menores;
- revisar mecanismos de compras e eventuais sistemas aleatórios.

Compliance deve ser validado novamente próximo ao lançamento, pois políticas podem mudar.

151. LAUNCH STRATEGY

Não lançar mundialmente imediatamente.

Preferir:

Internal Alpha

↓

Closed Alpha

↓

Technical Beta

↓

Regional Beta

↓

Soft Launch

↓

Global Launch

Cada etapa mede:

- estabilidade;
- performance;
- retention;
- economy;
- crash;
- server cost;
- monetization;
- exploitability.

152. SOFT LAUNCH

Objetivo não é ganhar dinheiro imediatamente.

Objetivo:

descobrir se o jogo funciona.

Métricas principais:

Tutorial completion

First-session completion

D1

D7

Session length

Crash-free sessions

Reconnect success

Economy stability
Combat engagement
Social interaction

153. GO / NO-GO

Cada milestone precisa de critérios objetivos.

Exemplo:

CRITICAL BUGS = 0
DATA DUPLICATION = 0
ECONOMIC EXPLOITS = 0
CRASH RATE <= threshold
RECONNECT SUCCESS >= threshold

Os thresholds devem ser definidos antes de cada fase.

154. RISCOS PRINCIPAIS

R1 — Scope explosion

Mitigação:

MVP rigoroso.

R2 — Content cost

Mitigação:

data-driven content.

R3 — Economy collapse

Mitigação:

telemetry + sinks + audit.

R4 — Botting

Mitigação:

server authority + behavior analysis.

R5 — Mobile UX

Mitigação:

mobile-first prototypes.

R6 — Performance

Mitigação:

profiling desde cedo.

R7 — Retention

Mitigação:

core loop antes de live ops.

R8 — Monetization

Mitigação:

monetização posterior ao core loop.

155. DECISÕES AINDA ABERTAS

Estas não devem ser escondidas.

PROJECT NAME

FINAL LORE

FINAL ART STYLE

FINAL CLASS COUNT

LEVEL CAP
DEATH PENALTY
OPEN WORLD PVP
WEIGHT SYSTEM
FINAL PROFESSIONS
FINAL COMBAT FORMULAS
FINAL XP CURVE
FINAL ECONOMIC VALUES
PREMIUM CURRENCY NAME
MONETIZATION PRICES
WORLD SIZE
NETWORK PROTOCOL DETAILS
SERVER SHARD STRATEGY

156. DECISÕES QUE PODEM SER DELEGADAS AO PROTÓTIPO

Não precisam ser decididas teoricamente:

Joystick vs tap
Attack speed
Cooldown length
TTK
Map density
Creature density
HUD layout
Camera distance
Particle density
Animation duration

Essas devem ser testadas.

157. REGRA PARA AGENTES DE IA

Agentes de desenvolvimento devem obedecer:

Não fazer

- inventar regra de produto;
- alterar arquitetura silenciosamente;
- adicionar dependência sem justificativa;
- apagar testes para fazer build passar;
- ignorar warnings importantes;
- modificar contratos sem atualizar consumidores;
- criar shortcuts que violem server authority.

Fazer

- procurar documentação;
- verificar código existente;
- manter testes;
- criar migrations;
- registrar decisões;
- implementar incrementalmente;
- adicionar testes antes/depois;
- documentar mudanças.

158. AGENT WORKFLOW

Para cada tarefa:

1. Read specification
2. Inspect repository
3. Identify impacted modules
4. Identify dependencies
5. Detect SPEC_GAP
6. Plan
7. Implement
8. Test
9. Run architecture tests
10. Run integration tests
11. Run build
12. Review security
13. Review concurrency
14. Review telemetry

15. Report evidence

159. AGENT REPORT

Toda tarefa deve terminar com:

STATUS:

PASS / PASS_WITH_FINDINGS / FAIL

CHANGED:

...

TESTS:

...

BUILD:

...

MIGRATIONS:

...

SECURITY:

...

CONCURRENCY:

...

SPEC_GAPS:

...

FOLLOW_UP:

...

160. PRIMEIRO BACKLOG REAL

EPIC 001 — Repository

- solution;
- projects;
- test projects;
- CI;
- Docker;
- documentation.

EPIC 002 — Identity

- account;
- login;
- session;
- token;
- character.

EPIC 003 — World

- map;
- tile;
- entity;
- movement;
- visibility.

EPIC 004 — Combat

- creature;
- target;
- attack;
- damage;
- death;
- loot.

EPIC 005 — Progression

- XP;

- level;
- attributes;
- skill.

EPIC 006 — Inventory

- item definition;
- item instance;
- inventory;
- equipment.

EPIC 007 — Quest

- quest;
- objectives;
- NPC;
- rewards.

EPIC 008 — Multiplayer

- synchronization;
- party;
- chat.

EPIC 009 — Economy

- wallet;
- transaction;
- trade;
- marketplace.

EPIC 010 — Production

- analytics;
- admin;
- monitoring;
- security;
- deployment.

161. PRIMEIRO OBJETIVO CONCRETO

O projeto não começa com:

“vamos construir um MMORPG”.

Começa com:

“Vamos provar que dois jogadores conseguem entrar em um pequeno mundo 2D, andar, encontrar uma criatura, lutar, matar, receber loot, evoluir e continuar exatamente de onde estavam depois de reconectar.”

Se isso for divertido:

expandimos.

Se isso não for divertido:

corrigimos antes de construir todo o resto.

162. REGRA FINAL DO PROJECT ETHER

O projeto deve sempre privilegiar:

Fun > Complexity

Clarity > Quantity

Discovery > Checklist

Player Agency > Automation

Long-term Economy > Short-term Monetization

Stable Core > Feature Count

Server Authority > Client Convenience

Content Quality > World Size

O objetivo não é construir o maior MMORPG.

O objetivo é construir um MMORPG em que o jogador queira descobrir:

“O que existe depois daquela montanha?”

e continuar jogando porque a resposta não estava escrita em uma lista de tarefas.

163. PRÓXIMA ETAPA

Este documento agora serve como **Product + Technical Baseline**.

A próxima especificação deve ser:

PROJECT ETHER — Blueprint v3.0

Com:

1. ERD completo;
2. schema PostgreSQL;
3. entidades C#;
4. value objects;
5. aggregates;
6. commands;
7. queries;
8. domain events;
9. APIs;
10. WebSocket protocol;
11. estados;
12. fórmulas matemáticas;
13. balance tables;
14. loot tables;
15. progression tables;
16. classes;
17. skills;
18. abilities;
19. creature definitions;
20. quest schema;
21. item schema;
22. economy schema;
23. marketplace transaction specification;
24. anti-cheat contracts;
25. telemetry event schema;
26. admin permissions;
27. migration strategy;
28. test matrix;
29. Docker/deployment;

- 30. CI/CD;
- 31. observability;
- 32. vertical-slice implementation backlog.

A partir do v3.0, o documento deixa de ser predominantemente Game Design e passa a ser uma especificação diretamente consumível por um agente de desenvolvimento.